



Oppimistehtävä 1 - Alikyselyt

Relaatiotietokannat

Jun Fengari, AG4042

Oppimistehtävä 1

Relaatiotietokannat, Jarkko Immonen

5.12.2025

Tietojenkäsittely

Sisältö

1	Ongelman analyysi	1
2	Toteutus	6
3	Erilaisia toteutustapoja	8
4	Reflektio ja tekoälyn käyttö	10
4.1	Reflektio	10
4.2	Tekoälyn Käyttö.....	10
Lähteet		12

1 Ongelman analyysi

1. Ketkä asiakkaista eivät ole koskaan lainanneet mitään?

Tarvittavat taulut: Asiakas, Laina

Yhteydet: Asiakkaan henkilötunnus toimii Lainaustaulussa asiakkaan tunnisteena.

Alikysely: `WHERE, NOT IN`

Vaiheet:

Ensimmäisenä tutkin, miten asiakkaat ja lainaukset yhdistyvät toisiinsa, ja kirjoitin pienen 'loogisen' pseudokoodipätkän, jonka pohjalla aloin pohtimaan miten yhdistetään ja tehdään tämä kysely.

Pseudokoodi: `select asiakas where asiakasID not in lainaus`

Ymmärsin tästä, että vaaditaan kaksi kyselyä: yksi joka hakee asiakkaat, ja toinen, joka hakee asiakkaiden lainaukset, ja sitten niitä verrataan 'ei olemassa' -operaatiolla.

- a) Pseudokoodin pohjalla kirjoitin tällöisen huonon koodin, ja kokeilin eri syntakseja, en ihan ymmärtänyt miten sen kuului toimia, mutta ajatus oli kuitenkin selkeä:

`SELECT * FROM asiakas`

`-- INNER JOIN lainaus ON lainaus.asiakas = asiakas.hloTunnus`

`WHERE asiakas.hloTunnus NOT IN (select 1 from lainaus.asiakas);`

- b) En osanut korjata sitä, niin kysyin ChatGPT:ltä mikä siinä on vikana, ja vika olikin tossa alikyselyssä, minun olisi selkeesti pitänyt ensin luoda sen alikyselyn loogisesti ennen kun aloin kirjoittamaan mitään kokonaisuutta.

✗ What's wrong

- You wrote `SELECT *`, but MySQL doesn't allow `*` inside a subquery that's supposed to return one column for comparison.
 - `NOT IN` (or `IN`) expects a single-column list — something like `(1, 2, 3, 4)` or `(SELECT column_name FROM table)`.
 - `SELECT *` tries to return all columns from the table, which doesn't make sense in that context.
- You also wrote `lainaus.asiakas` in the `FROM` clause like `FROM lainaus.asiakas`.
 - That syntax means "the table `asiakas` inside the schema `lainaus`" — not what you mean.
 - You probably meant "the column `asiakas` from the table `lainaus`," which should be written as `SELECT asiakas FROM lainaus`.

Tässä on oikein kirjoitettu lause:

`SELECT * FROM asiakas`

`WHERE asiakas.hloTunnus NOT IN (SELECT asiakas FROM lainaus);`

- c) Viimeisenä vaiheena siistin koodin näyttämään relevantin tuloksen. Valmis koodi löytyy toteutuksesta.

2. Ketkä asiakkaat ovat lainanneet ainakin yhden kirjan samasta luokasta kuin "Kuolema Niilillä", mutta eivät ole koskaan lainanneet "Kuolema Niilillä" -teosta?

Tarvittavat taulut: Nimike, Laina, Asiakas, Nide, Laina

Yhteydet: Nimeke (luokitus) -> Nide (yhdistää) -> Laina

`nimeke.nimeke-nide.nimeke -> nide.nide-lainausnide.nide -> lainausnide.lainaus -> lainaus.lainaus -> lainaus.asiakas -> asiakas.hloTunnus`

Alikysely: `WHERE`

Vaiheet:

Aloitin tämän tehtävän tutkimalla tietokantaa ja yhteyksiä, ja miettimään, mitä asioita piti tehdä erikseen, esimerkiksi taulujen liittäminen ja erilaisten ehtojen täyttämistä.

Tutkin eri kyselyiden avulla muistinvirkistysenä, miten saadaan eri tietoja näkyviin.

Kuvaan alla jokaisen vaiheen ja sen koodipätkän, joka siitä vaiheesta syntyi. Toteutuksessa näkyy miten nämä osiot ovat yhdistyneet kokonaisuudeksi.

- a) Ensin haetaan Kuolema Niilillä luokan: `SELECT * WHERE nimeke.nimi = 'Kuolema Niilillä' -> SELECT nimeke.luokitus FROM nimeke WHERE nimeke.nimi = 'Kuolema Niilillä'`
- b) Sitten liitetään taulut yhteen käyttäen yhteyksiä kuvattu yllä yhteydet osiossa:
`SELECT * FROM nimeke
 INNER JOIN nide ON nide.nimeke = nimeke.nimeke
 INNER JOIN lainausnide ON nide.nide = lainausnide.nide
 INNER JOIN lainaus ON lainausnide.lainaus = lainaus.lainaus
 INNER JOIN asiakas ON lainaus.asiakas = asiakas.hloTunnus`
- c) Yhdistetään a:n kysely b:n alle käyttäen WHERE:
`WHERE nimeke.luokitus = (SELECT nimeke.luokitus FROM nimeke WHERE nimeke.nimi = 'Kuolema Niilillä')`
- d) Tämä palauttaa oikean luokan, mutta pitää vielä poistaa Kuolema Niilillä, jonka voidaan yhdistää AND:illä c:n loppuun: `AND nimeke.nimi != 'Kuolema Niilillä'`
- e) Paikkasin alusta select *, asiakkaan nimillä siistiin sarakkeeseen, ja otin pois tuplat: `SELECT DISTINCT CONCAT(asiakas.etunimi, ' ', asiakas.sukunimi) AS asiakas_nimi`
- f) Järjestin tuloksen aakkosjärjestykseen etunimen mukaan: `ORDER BY asiakas_nimi;`

3. Montako eri luokitusta jokaisen asiakkaan lainoihin sisältyy?

Ymmärsin, että halutaan tulos, jossa on asiakkaan nimi ja toiseen sarakkeeseen määrä erilaisista lainausluokituksista, joita kukin asiakas on lainannut.

Pseudokoodi: select asiakasnimi ja eriluokitukset from asiakastaulu + select distinct luokitukset + count

Tarvittavat taulut: Nimike, Lainaus, Asiakas, Nide, LainausNide

Yhteydet: Koska asiakkaat pitää yhdistää taas luokituksiin, voidaan käyttää 2. tehtävän joinoja.

Eli: nimeke.nimeke-nide.nimeke -> nide.nide-lainausnide.nide -> lainausnide.lainaus ->

lainaus.lainaus -> lainaus.asiakas -> asiakas.hlotunnus

Alikysely: `SELECT`

Vaiheet:

- a) Kopioin alikyselyn dokumentaatiosta (Jarkko Immonen, ei pvm.) alikysely select osiossa esimerkin, ja aloin kokeilemaan miten se voisi toimia.
`SELECT hloTunnus, etunimi, sukunimi, (
 SELECT COUNT(*) FROM Puhelin p2
 WHERE a.hloTunnus=p2.asiakas
) AS puhLkm
 FROM Asiakas a`
- b) Yritin yhdistää tuotokseni joinin kanssa, mutta jotain vikaa oli syntaksissa:
`SELECT hloTunnus, etunimi, sukunimi, (SELECT DISTINCT COUNT(*) FROM nimeke)
 AS unique_category FROM Asiakas
 +
 INNER JOIN nide ON nide.nimeke = nimeke.nimeke
 INNER JOIN lainausnide ON nide.nide = lainausnide.nide
 INNER JOIN lainaus ON lainausnide.lainaus = lainaus.lainaus
 INNER JOIN asiakas ON lainaus.asiakas = asiakas.hloTunnus`

Kysyin tekoälyltä miten voisin korjata tämän toimivaksi, mutta se ei osanut ratkoa ongelmaa kunnolla, ja haluisi mielummin käyttää GROUP toimintoa.

Työstin koodia kunnes huomasin että minulla on joinit väärin.

- c) Sain koodin tuottamaan tuloksen tällä, mutta luokitusten määrä oli kaikissa sama.
`SELECT asiakas.hloTunnus, asiakas.etunimi, asiakas.sukunimi,`

```
(SELECT COUNT(DISTINCT nimeke.luokitus) FROM LainaUs Ins
  INNER JOIN asiakas on Ins.asiakas = asiakas.hloTunnus
  INNER JOIN lainausnide on Ins.lainaus = lainausnide.lainaus
  INNER JOIN nide on lainausnide.nide = nide.nide
  INNER JOIN nimeke on nide.nimeke = nimeke.nimeke
) AS unique_categories FROM Asiakas
```

d) Kysyn taas tekoälyltä mikä ongelma on:

You're selecting from `Asiakas` **outside** the parentheses,
and **again** joining `Asiakas` **inside** the subquery.

But the inner `asiakas` is its *own table reference*, separate from the outer one — they aren't connected in any way.

 What's missing

You need to make the subquery **correlated** — meaning it depends on the *outer* row's customer.

Right now, your subquery doesn't know *which customer* from the outer query it's supposed to calculate for.

 The fix (correlate it properly)

You don't need to join `Asiakas` inside the subquery at all —
you can simply refer to the outer query's alias (like `a.hloTunnus`):

Eli ei voinutkaan käyttää samalla tavalla joineja kuten suunnitteluvaiheessa luulin, vaan piti käyttää aliasta ja poista asiakkaan tuplataulun. Näiden korjausten jälkeen sain oikean tuloksen.

4. Keillä asiakkailla on enemmän lainoja kuin Kari Mäkelällä? Tähän liittyy extra-vaatimus: kyselyssä tulee esiintyä ehdot etunimi="Kari" ja sukunimi="Mäkelä".

Tarvittavat taulut: Asiakas (nimi), LainaUs (hetu)

Yhteydet: asiakas.hloTunnus -> lainaus.asiakas

Alikysely: `HAVING`

Vaiheet:

Suunnitelma olisi ensin laskea asiakkaiden lainausmäärät ja valita niistä vain ne, jotka ovat enemmän kuin Karin määrä. Alikyselyyn tulee Karin lainausten lasku, ja niitä voidaan verrata pääkyselyssä.

Pseudokoodi: `Select asiakkaat having > (select count karin lainaukset)`

- a) Tutustuin ensin miten lainaukset on merkitty tauluun, käytin `SELECT DISTINCT asiakas FROM lainaus`, ja huomasin että hetut on siellä moneen kertaan.
- b) Sitten lähdin koodaamaan ensin alikyselyä, ja ensin valitsin Karin sieltä:
`SELECT * FROM LainaUs`
`INNER JOIN Asiakas ON Asiakas.hloTunnus = LainaUs.asiakas`
`WHERE etunimi LIKE 'Kari' AND sukunimi LIKE 'Mäkelä'`
- c) Lisäsin sitten tähän group by ja laskun, jolla sain oikean määrän Karin lainauksia:
`SELECT COUNT(hloTunnus) FROM LainaUs`
`INNER JOIN Asiakas ON Asiakas.hloTunnus = LainaUs.asiakas`
`WHERE etunimi LIKE 'Kari' AND sukunimi LIKE 'Mäkelä'`
`GROUP BY hloTunnus`

- d) Sitten tein saman muille asiakkaille:
`SELECT COUNT(hloTunnus) FROM Lainaus
 INNER JOIN Asiakas ON Asiakas.hloTunnus = Lainaus.asiakas
 GROUP BY hloTunnus`
- e) Sitten yhdistin ne käyttäen having:
`SELECT COUNT(hloTunnus) AS lkm FROM Lainaus
 INNER JOIN Asiakas ON Asiakas.hloTunnus = Lainaus.asiakas
 GROUP BY hloTunnus HAVING lkm >
 (SELECT COUNT(hloTunnus) FROM Lainaus
 INNER JOIN Asiakas ON Asiakas.hloTunnus = Lainaus.asiakas
 WHERE etunimi LIKE 'Kari' AND sukunimi LIKE 'Mäkelä'
 GROUP BY hloTunnus)`
- f) Ja sitten korjasin sen näyttämään tulokset oikein.

5. Mitä nimikkeitä kukin kirjailija on kirjoittanut? Listaa kukin kirjailija kerran ja näytä hänen kirjoittamiensa kirjojen nimet sarakkeessa "nimekkeet" pilkulla erotettuna toisistaan.

Tarvittavat taulut: Kirjailija, Nimeke

Yhteydet: kirjailija.idKirjailija -> nimeke.kirjailija

Alikysely: `SELECT`

Vaiheet:

En osanut heti hahmoittaa täysin miten tämä toimisi, niin aloin ensin kyseilyllä kokeilemaan minkälaista tulosta saisi.

- a) Ensin katoin miten voisin saada kirjailijoiden teokset tuloksena, kokeilin hakemalla tiettyä kirjailijaa:
`SELECT idKirjailija, nimi FROM kirjailija
 INNER JOIN nimeke ON kirjailija.idKirjailija = nimeke.kirjailija
 WHERE idKirjailija='2';`
- b) Tämän jälkeen en saanut käsitystä että mikä tän kyselyn yleinen struktuuri voisi olla, niin kysyin ChatGPT:ltä apua hahmoittamaan loogista ajattelua tähän tehtävään.

B Path 2 — Subquery per author

You again start with `kirjailija`, but instead of joining directly, you say:

"For this one author (the current row), go look into the Nimeke table and collect all titles where the kirjailija matches this one's ID."

Conceptually:

vbnet

 Copy code

Outer query: `loop` through `each` author
 Inner query: look up all books `WHERE kirjailija_id = this author's id`
 Return that list of names as one `text` field

That's what a **correlated subquery** does — it runs once per outer row, using a value (the author's ID) to look up matching details.

🔧 Step 5: Why this works

So, no matter which structure you use:

- You always have **one table driving the result** (the “one” side — `Kirjailija`)
- And **one table being summarized or collected** (the “many” side — `Nimeke`)
- You always connect them through the **foreign key** (`idKirjailija` ↔ `kirjailija`)

Everything else (JOINS, subqueries, GROUP_CONCAT) is just *syntax* that implements that mental picture.

🧠 The mental recipe

1. Decide what one row represents.
 - “Each author” → start from `Kirjailija`
2. Decide what you want to collect or summarize.
 - “All their book names” → that’s data from `Nimeke`
3. Figure out how they connect.
 - `kirjailija.idKirjailija = nimeke.kirjailija`
4. Pick your method:
 - JOIN + GROUP BY (all in one query)
 - or correlated subquery (outer = author, inner = look up titles)

- c) Lisäsin sitten siihen GROUP_CONCAT syntaksin ensin:
`SELECT GROUP_CONCAT(nimi SEPARATOR ' ') AS nimekkeet FROM kirjailija`
- d) Tiesin että tämä äskeinen kysely on se sisällä oleva äskeisen tekoälykeskustelun perusteella, ja sitten aloin katsomaan miten saisin ulomman kyselyn valmiiksi, eli käytännössä kirjailijan tiedot.
`SELECT idKirjailija, etunimi, sukunimi, (alikysely tähän) AS nimekkeet FROM kirjailija`
- e) Sitten muistin edellisestä tehtävästä että alikysely kytkeyty WHEREin avulla, joten päähkäilin sen syntaksin ja aliaksien kanssa kunnes sain halutun lopputuloksen.

2 Toteutus

1. `SELECT CONCAT(asiakas.etunimi, ' ', asiakas.sukunimi) AS asiakas_nimi, asiakas.hloTunnus FROM asiakas WHERE asiakas.hloTunnus NOT IN (SELECT asiakas FROM lainaus);`

Alikysely on itsenäinen.

Mielestäni tämä oli helppo loogisesti ajatella, koska siinä on selkeesti kaksi kyselyä, jota tarkistetaan ’toisia vastaan’. Ainoa haaste oli muistella syntakseja.

2. `SELECT DISTINCT CONCAT(asiakas.etunimi, ' ', asiakas.sukunimi) AS asiakas_nimi FROM nimeke INNER JOIN nide ON nide.nimeke = nimeke.nimeke INNER JOIN lainausnide ON nide.nide = lainausnide.nide INNER JOIN lainaus ON lainausnide.lainaus = lainaus.lainaus INNER JOIN asiakas ON lainaus.asiakas = asiakas.hloTunnus WHERE nimeke.luokitus = (SELECT nimeke.luokitus FROM nimeke WHERE nimeke.nimi = 'Kuolema Niilillä') AND nimeke.nimi != 'Kuolema Niilillä' ORDER BY asiakas_nimi;`

Alikysely on itsenäinen.

Tämä oli mielestäni tosi vaikea, työstettiin tätä yhdessä porukassa ja lopulta ymmärsin, miten

pystyy järjestelmällisesti ajattelemaan jokaista vaihetta läpi ja kokeilemaan miten eri vaiheet yhdistyvät. Nyt kun katselen valmista koodia, niin huomaa että siinä on ymmärrettävä ja looginen kulku.

3.

```
SELECT a.hloTunnus, a.etunimi, a.sukunimi,
      (SELECT COUNT(DISTINCT nimeke.luokitus) FROM LainaUs Ins
      INNER JOIN lainausnide on Ins.lainaUs = lainausnide.lainaUs
      INNER JOIN nide on lainausnide.nide = nide.nide
      INNER JOIN nimeke on nide.nimeke = nimeke.nimeke
      WHERE a.hloTunnus = Ins.asiakas
      ) AS unique_categories
FROM Asiakas a
```

Alikysely on kytketty, ei toimi itsenäisesti.

Tää oli erittäin vaikea, jouduin käyttämään tekoälyä selittämään ensin, mitä kysymyksellä haettiin, ja sitten monessa vaiheessa se ei osanut täsmentää ongelmaa, niin piti tehdä monta kokeilua ja uudelleenkirjoitusta. Se tapa millä kytkettiin kysely (käyttäen WHERE joinin sijasta) oli hankalin konsepti. Join myös yleisesti jäi aiemmasta kurssista vähän epäselväksi, niin ne tuottavat minulle pieniä loogisia ongelmia.

4.

```
SELECT COUNT(hloTunnus) AS InsIkm, etunimi, sukunimi FROM LainaUs
INNER JOIN Asiakas ON Asiakas.hloTunnus = LainaUs.asiakas
GROUP BY hloTunnus HAVING InsIkm >
      (SELECT COUNT(hloTunnus) FROM LainaUs
      INNER JOIN Asiakas ON Asiakas.hloTunnus = LainaUs.asiakas
      WHERE etunimi LIKE 'Kari' AND sukunimi LIKE 'Mäkelä'
      GROUP BY hloTunnus)
ORDER BY sukunimi
```

Alikysely on itsenäinen.

Tämä oli aiempien tehtävien jälkeen taas vähän helpompi kysely ja ainoat haasteet olivat pienet syntaksivirheet.

5.

```
SELECT idKirjailija, etunimi, sukunimi, (
      SELECT GROUP_CONCAT(nimi SEPARATOR ', ') FROM kirjailija
      INNER JOIN nimeke ON kirjailija.idKirjailija = nimeke.kirjailija
      WHERE krj.idKirjailija = nimeke.kirjailija
      GROUP BY idKirjailija
      ) AS nimekkeet FROM kirjailija AS krj
```

Alikysely on kytketty, ja se yhdistyy where osassa pääkyselyn idKirjailijaan.

Tässä tehtävässä oli haastavaa hahmoittaa kokonaisuutta, toisin kun itsenäisissä kyselyissä jossa molemmat osiot olivat selkeitä. Edelleen tuli haasteita syntaksien muistelussa ja aliaksien käytössä.

3 Erilaisia toteutustapoja

Valittu kysely:

```
SELECT CONCAT(asiakas.etunimi, ' ', asiakas.sukunimi) AS asiakas_nimi, asiakas.hloTunnus FROM asiakas
WHERE asiakas.hloTunnus NOT IN (SELECT asiakas FROM lainaus);
```

Valitsin helpoimman ja lyhyimmän kyselyn, tämä on jo IN alikysely.

- **Käsittelytapa:** IN tarkistaa löytyykö joku arvo tietystä joukosta. Tässä tilanteessa verrataan että löytyykö lainaustaulusta asiakkaan henkilötunnusta, ja koska kyselyssä on NOT IN, ne hetut mitkä EI löydy, otetaan tulosjoukkoon mukaan.
- **Tehokkuus:** IN toimii hyvin pienille joukoille, mutta saattaa olla hidas ja raskas isommille data määriille. Explain toiminnolla nähdään että tämä kysely käy todennäköisesti 100% molemmista tauluista läpi. Rivejä se käy asiakas taulussa ~37 läpi, ja lainaus taulussa ~7 läpi. Kun tarkistetaan profiilit, niistä voi nähdä että tämä kysely vei 0.00563350 sekunttia.

EXISTS-alikysely:

```
SELECT CONCAT(asiakas.etunimi, ' ', asiakas.sukunimi) AS asiakas_nimi, asiakas.hloTunnus FROM asiakas
WHERE NOT EXISTS (SELECT 1 FROM lainaus WHERE asiakas.hloTunnus = lainaus.asiakas);
```

- **Käsittelytapa:** EXISTS tarkistaa löytyykö joku arvo tietystä joukosta, ja toisin kun IN, palauttaa totuusarvon. Tässä tapauksessa jos alikysely palauttaa totta, sitä ei oteta tulosjoukkoon mukaan, koska käytetään NOT EXISTS.
- **Tehokkuus:** EXISTS on yleensä tehokkaampi kun IN kun käsitellään isompia tauluja, ja se pystyy lopettamaan aiemmin tietyissä tapauksissa. Explain komennolla näyttää saman tuloksen kun ylempi NOT IN kysely. Profiileissa nähdään että kysely vei 0.00019425 sekunttia, joka on nopein tulos.

JOIN:

```
SELECT CONCAT(asiakas.etunimi, ' ', asiakas.sukunimi) AS asiakas_nimi, asiakas.hloTunnus, lainaus.asiakas
FROM asiakas
LEFT JOIN Lainaus ON Asiakas.hloTunnus = Lainaus.asiakas
WHERE lainaus.asiakas IS NULL
```

- **Käsittelytapa:** LEFT JOIN liittää Lainaus taulun Asiakas tauluun, ja sisällyttää Null values, joita nimenomaan tarvitaan tässä kyselyssä. Eli se yhdistää kaikki asiakastaulun rivit lainaustaulun ehtoon, ja jos jollakin yhdistettävällä rivillä ei ole arvoa, se lisää siihen Null. Lopuksi WHERE ehto valitsee nämä Null tulokset mitä haetaan.
- **Tehokkuus:** LEFT JOIN joutuu yhdistämään molemmat taulut, joka saattaa olla teho-intensiivisempi varsinkin isoissa tulosjoukoissa, mutta tehoa voi parantaa kunnollisella indeksoinnilla. Explain komento näyttää jälleen samat tulokset kun aiemmin. Profiileissa näkyy että kysely vei 0.00140725 sekunttia, eli keskinopein näistä kyselyistä.

Oma Kysely

Kuinka monta nidettä jokaisella kirjalla on, ja millä kirjoilla on 3 tai enemmän nidettä?

Tarvittavat Taulut: Nide, Nimeke

Yhteydet: nide.nimeke -> nimeke.nimeke

Toteutus:

Ensin toteutin JOIN:

```
SELECT COUNT(nide.nimeke) AS niteidenlkm, nimeke.nimi FROM nimeke
INNER JOIN nide ON nide.nimeke = nimeke.nimeke
GROUP BY nide.nimeke
HAVING niteidenlkm >= 3;
```

Kasittelytapa: Nide liitetään nimeke tauluun jotta nähdään kirjan nimet (nimikkeistä) ja uusi sarake jossa lasketaan niteiden määrät (niteistä), joista sitten valitaan HAVING:illä vain kirjat joilla on enemmän kuin 3 nidettä.

Sitten kokeilin tehdä itsenäistä alikyselyä, mutta se olikin helpompi tehdä kytkettynä alikyselynä:

```
SELECT (SELECT COUNT(*) FROM nide
WHERE nide.nimeke = nimeke.nimeke) AS niteidenlkm, nimeke.nimi
FROM nimeke HAVING niteidenlkm >= 3;
```

Käsittelytapa: Alikysely SELECT osassa laskee kaikkien kirjojen niteet, ja sitten se yhdistyy WHERE osassa vastaavanlaisesti kun aiemmin, ja pääkyselyssä sitten valitaan näytettävät sarakkeet ja lisätään HAVING ehto.

IN toteutuksessa kohtasin ongelmia koska ymmärsin että IN käsittelee vain yhtä saraketta. Kysyin tekoälyltä miten tämän voisi toteuttaa, mutta se antoi mulle kyselyitä jossa oli useampia alikyselyitä.

why doesn't this work? I need both columns to show as it did in the previous task:
 SELECT nimeke.nimi, niteidenlkm FROM nimeke WHERE
 nimeke IN (SELECT nide.nimeke, COUNT(nide.nimeke) AS
 niteidenlkm FROM nide GROUP BY nide.nimeke HAVING
 COUNT(*) >= 3);

Excellent question — and you're so close!

This is a very common conceptual mix-up when moving from a `JOIN` to an `IN` subquery.

Let's break down **why it doesn't work** and how to fix it — conceptually first.

The issue is in this line:

```
sql
SELECT nide.nimeke, COUNT(nide.nimeke) AS niteidenlkm
```

Here, your **subquery** returns **two columns**, but **IN** can only work with **one column** — it checks membership like this:

"Is this single value in that list of single values?"

So conceptually, this doesn't make sense:

"Is the *whole row* (name + count) in that list?"

SQL doesn't know how to compare two columns against one condition.

Kokeilin tässä vaiheessa myös eri kyselyitä, jotka täyttäisivät kaikki vaaditut kyselymuodot, mutta niissä tuli jälleen sama ongelma vastaan.

Päädyin näyttämään seuraavissa kyselyissä vain kirjojen nimet, ja jättämään tarkat niteiden lukumäärät pois, sillä sitä ei pystynyt toteuttamaan IN ja EXISTSillä ilman monta alikyselyä.

Näytä kirjat joilla on enemmän kuin 3 nidettä.

```
SELECT nimeke.nimi FROM nimeke
WHERE nimeke IN
(SELECT nide.nimeke FROM nide
GROUP BY nide.nimeke
HAVING COUNT(*) >= 3);
```

Kasittelytapa: Itsenäinen alikysely. Pääkyselyssä valitaan kirjan nimet nimikke taulusta ja alikyselyssä lasketaan ja valitaan kaikki niteiden nimikkeet jotka täyttävät HAVING ehdon. Nämä ovat liitetty yhteen niin että pääkysely palauttaa nimet jotka vastaa WHERE osion ehtoon (alikäyselyn nide.nimeke = pääkäyselyn nimeke.nimeke).

EXISTS toteutus:

```
SELECT nimeke.nimi FROM nimeke
WHERE EXISTS
(SELECT 1 FROM nide WHERE nide.nimeke = nimeke.nimeke
GROUP BY nide.nimeke
HAVING COUNT(*) >= 3);
```

Käsittelytapa: Kytketty alikysely. Pääkysely valitsee kirjan nimet nimeke taulusta, jonka sitten alikysely käy läpi exists ehdon kautta, eli palauttaa totta kaikille kirjoille joilla on 3 tai enemmän nidettä.

4 Reflektio ja tekoälyn käyttö

4.1 Reflektio

Opin tästä oppimistehtävästä todella paljon. Huomasin tätä tehdessä, että tämä kurssi on ollut paljon laajempi ja haastavampi kuin aiemmin suoritettu insinööripuolen hakukurssi. Aiheet olivat samat, mutta näiden tehtävien kautta perehdyin alikyselyihin paljon syvällisemmin.

Opin seuraavia asioita:

- Alikyselyitä on itsenäisiä ja kytkettyä. Opin niiden syntaksilliset ja toiminnalliset erot, ja opin myös vähän hahmoittamaan, milloin kuuluisi käyttää kumpaakin.
- Alikyselyitä voi laittaa pääkyselyn eri osiin (Select, Where, Having) ja opin miten ne eroaa toisistaan.
- Samaa alikyselyä voi toteuttaa monella eri tavalla, ja alikyselyitä voi tehdä myös JOIN-ratkaisulla.

Tämä oppimistehtävä muistutti ja opetti paljon muista SQL-asioista, kuten JOINista, joka oli jäänyt vähän epäselväksi edellisestä kurssista.

Missä tilanteissa käyttäisit mieluummin JOIN-ratkaisua kuin alikyselyä?

Huomasin viimeisessä ”Oma Kysely”-tehtävässä, että alikyselyitä käytetään usein tulosten suodattamiseen, ja niiden kanssa on vaikeata näyttää useamman taulun tietoja ilman, että menee liian monimutkaiseksi. Ymmärsin, että JOIN on selkeämpi ja luettavampi tapa tehdä kyselyitä.

Eri kyselyn muodot myös paljastivat että JOIN on mahdollisesti tehokkaampi, jos sitä käytetään oikein.

Mikä vaihe vei eniten aikaa tai aiheutti eniten virheitä?

Oppimistehtävä oli hyvin haastava, ja tein melkein kaikissa vaiheissa virheitä.

Edellisen kurssin opetustapa ja tehtävänanto olivat niin erilaista, että jouduin uudelleen oppimaan ja muistuttamaan itselleni melkein kaiken.

Koin ensimmäisessä osiossa vaikeuksia suunnitella etukäteen miten kysely voisi toimia, sillä muistini tietokantojen oppinnoista oli heikko, ja kyseinen tietokanta oli minulle uusi. Onnistuin paremmin trail-and-error -tyylillä, ja siitä johtuen päätin sisällyttää ensimmäisten tehtävien alle ajatteluni ja kyselyiden muodostamisprosessin.

Myöhemmin, riittävän työskentelyn jälkeen aloin ymmärtää tietokannan struktuuria paremmin, ja muistella suunnilleen, mitkä osat vaaditaan toimivaan kyselyyn.

Syntaksivirheitä tuli myös jatkuvasti; melkein kahden vuoden aikana olen käyttänyt niin monta eri kieltä että SQL:n syntaksi oli osittain unohtunut.

Vertailin oppimistehtäväni lopputulostani opiskelutovereiden kanssa ja huomasin, että koodini näyttää hieman erilaiselta, varsinkin aliaksien osalta. Koin aliaisten käytön tosi hämmentäväksi, ja käytin niitä vain silloin kun ne olivat tarpeellisia, mikä aiheutti myös omanlaista haastetta.

Kyselyitä oli helpompi hahmoittaa kun pystyin näkemään selkokielisesti, mikä liittyy mihinkäkin; yksittäiset kirjaimet tai kirjainyhdistelmät sekoittivat hahmoittamista liikaa, vaikka se onkin hyvien käytäntöjen mukaista.

Käytin tämän oppimistehtävän tekemiseen kaksi viikkoa, yhteensä noin 20 tuntia.

4.2 Tekoälyn Käyttö

Käytin tekoälyä tässä oppimistehtävässä tehtävien hahmottamiseen, oman koodin vianetsintään ja korjausehdotuksiin, sekä lopullisen oman tuotoksen tarkistukseen.

Dokumentoin jokaiseen tehtävän sisältöön tekoälyn käytöstä kommentteja ja näytönkaappaukset

relevantteihin vaiheisiin.
Kuvaan alla muita vaiheita, jossa käytin tekoälyä:

Tehokkuuksia verratessa käytin tekoälyä selventämään, miten IN, EXISTS ja normaali JOIN eroavat. ChatGPT ohjeisti seuraavaksi, jonka pohjalta sain paremman ymmärryksen tehokkuudesta:

Tehokkuus

- MySQL can **stop early**: it doesn't need to check all rows inside the subquery, just one.
- That makes it often **faster** than `IN` when the tables are big.
- It also **ignores NULLs safely**, so it doesn't get tripped up like `NOT IN`.

Summary:

`EXISTS` is usually more efficient than `IN` for larger tables and safer when NULLs exist. It checks "Does at least one matching row exist?" rather than comparing lists.

Tehokkuus

- MySQL has to **join both tables completely**, which can be more data-heavy.
- But with **proper indexes** (like on `lainaus.asiakas`), the optimizer can make it very fast.
- Sometimes MySQL automatically **rewrites a `NOT EXISTS` query internally as a `LEFT JOIN`** if it's faster — so in practice, performance can be *nearly identical*.

Summary:

`LEFT JOIN ... IS NULL` is easy to understand and often efficient if tables are indexed, but it can do more work if the join produces many intermediate rows.

In plain language — how they compare

Method	How it thinks	Performance	Notes	
NOT IN	"Is this value in that list?"	Slower on large lists; can break with <code>NULL</code>	Easy to read, fine for small tables	
NOT EXISTS	"Does a matching row exist?"	Often fastest and safest	Stops early, ignores <code>NULL</code> issues	
LEFT JOIN ... IS NULL	"Join everyone and see who doesn't match."	Depends on indexes; can be equal to <code>EXISTS</code>	Good readability; very common pattern	

Tehokkuuden seurannassa tekoäly myös ehdotti "profile"-toimintoa, jolla pystyi katsomaan kyselyiden suoritusnopeutta. Tulokset vaihtelivat, ja kyselyt olivat niin pieniä, että tehokkuus ei ole tässä mittakaavassa eronut merkittävästi.

Pyysin ChatGPT:tä generoimaan myös esimerkkikysymyksiä omaan kyselyyni. Muita ehdotusta ei voinut toteuttaa IN ja EXISTS tavoilla, joten jouduin pyytämään ideoita lisää-osiin kyselyyni:

Absolutely — here are several "jatkokysymys" ideas that keep the same *theme* ("nimeke" and "nide") but shift the focus slightly so they're perfect for simple `IN` / `EXISTS` queries 🗨️

- Based on *existence* rather than *counting***
- Mitkä kirjat eivät ole saaneet yhtään nidettä?
 - Mitkä kirjat ovat saaneet ainakin yhden niteen?
 - Mitkä kirjat ovat sellaisia, että niistä on tehty useampi nide kuin jostain toisesta kirjasta?
 - Mitkä kirjat ovat sellaisia, että niistä on tehty nide, jota ei ole vielä lainattu kertaakaan?
 - Mitkä kirjat kuuluvat samaan luokkaan kuin joku tietty kirja (esim. "Kuolema Niilillä")?

Oppimistehtävän lopussa tarkistutin kaikkien kirjoitusosien kirjoitusvirheet ja kieliasuvirheet tekoälyn avulla.

Lähteet

Käytin lähteinä kurssin alikyselydokumentaatiotiedostoa sekä omia muistiinpanoja edellisestä kurssista.

Jarkko Immonen. (ei pvm.). *4_alikyselyt*. Noudettu 14. marraskuuta 2025, osoitteesta

https://tiko.jamk.fi/~imjar/reltk/4_alikyselyt.html

